UNIT 3

File System

File Concept

Computer store information in storage media such as disk, tape drives, and optical disks. The operating system provides a logical view of the information stored in the disk. This logical storage unit is a file.

The information stored in files are non-volatile, A file is named collection of related information that is stored on physical storage.

Data cannot be written to a computer unless it is written to a file. A file, in general, is a sequence of bits, bytes, lines, or records defined by its owner or creator. The file has a structure defined by its owner or creator and depends on the file type.

- Text file – It has a sequence of characters.
- Image file – It has visual information such as photographs, vectors art and so on.
- Source file – It has subroutines and function that are compiled later.
- Object file – It has a sequence of bytes, organized into bytes and used by the linker.
- Executable file – The binary code that the loader brings to memory for execution is stored in an exe file.

Operations on the File

- A file is a collection of logically related data that is recorded on the secondary storage in the form of sequence of operations. The content of the files are defined by its creator who is creating the file. The various operations which can be implemented on a file such as read, write, open and close etc. are called file operations. These operations are performed by the user by using the commands provided by the operating system.
- **1.Create operation:**
- This operation is used to create a file in the file system. It is the most widely used operation performed on the file system. To create a new file of a particular type the associated application program calls the file system. This file system allocates space to the file. As the file system knows the format of directory structure, so entry of this new file is made into the appropriate directory.
- **2. Open operation:**
- This operation is the common operation performed on the file. Once the file is created, it must be opened before performing the file processing operations. When the user wants to open a file, it provides a file name to open the particular file in the file system. It tells the operating system to invoke the open system call and passes the file name to the file system.
- **3. Write operation:**
- This operation is used to write the information into a file. A system call write is issued that specifies the name of the file and the length of the data has to be written to the file.

Whenever the file length is increased by specified value and the file pointer is repositioned after the last byte written.

- **4. Read operation:**
- This operation reads the contents from a file. A Read pointer is maintained by the OS, pointing to the position up to which the data has been read.
- **5. Re-position or Seek operation:**
- The seek system call re-positions the file pointers from the current position to a specific place in the file i.e. forward or backward depending upon the user's requirement. This operation is generally performed with those file management systems that support direct access files.
- **6. Delete operation:**
- Deleting the file will not only delete all the data stored inside the file it is also used so that disk space occupied by it is freed. In order to delete the specified file the directory is searched. When the directory entry is located, all the associated file space and the directory entry is released.
- **7. Truncate operation:**
- Truncating is simply deleting the file except deleting attributes. The file is not completely deleted although the information stored inside the file gets replaced.
- **8. Close operation:**
- When the processing of the file is complete, it should be closed so that all the changes made permanent and all the resources occupied should be released. On closing it deallocates all the internal descriptors that were created when the file was opened.
- AD
- **9. Append operation:**
- This operation adds data to the end of the file.
- **10. Rename operation:**
- This operation is used to rename the existing file.

Types of files

The following are basic types of files

Item	Description
regular	Stores data (text, binary, and executable)
directory	Contains information used to access other files
special	Defines a FIFO (first-in, first-out) pipe file or a physical device

Regular files

Regular files are the most common files and are used to contain data. Regular files are in the form of text files or binary files:

Text files

Text files are regular files that contain information stored in ASCII format text and are readable by the user. You can display and print these files. The lines of a text file must not contain NUL characters, and none can exceed {LINE_MAX} bytes in length, including the newline character.

The term *text file* does not prevent the inclusion of control or other nonprintable characters (other than NUL). Therefore, standard utilities that list text files as inputs or outputs are either able to process the special characters or they explicitly describe their limitations within their individual sections.

Binary files

Binary files are regular files that contain information readable by the computer. Binary files might be executable files that instruct the system to accomplish a job. Commands and programs are stored in executable, binary files. Special compiling programs translate ASCII text into binary code.

Text and binary files differ only in that text files have lines of less than {LINE_MAX} bytes, with no NUL characters, each terminated by a newline character.

Directory files

Directory files contain information that the system needs to access all types of files, but directory files do not contain the actual file data. As a result, directories occupy less space than a regular file and give the file system structure flexibility and depth. Each directory entry represents either a file or a subdirectory. Each entry contains the name of the file and the file's index node reference number (*i-node number*). The i-node number points to the unique index node assigned to the file. The i-node number describes the location of the data associated with the file. Directories are created and controlled by a separate set of commands.

Special files

Special files define devices for the system or are temporary files created by processes. The basic types of special files are FIFO (first-in, first-out), block, and character. FIFO files are also called *pipes*. Pipes are created by one process to temporarily allow communication with another process. These files cease to exist when the first process finishes. Block and character files define devices.

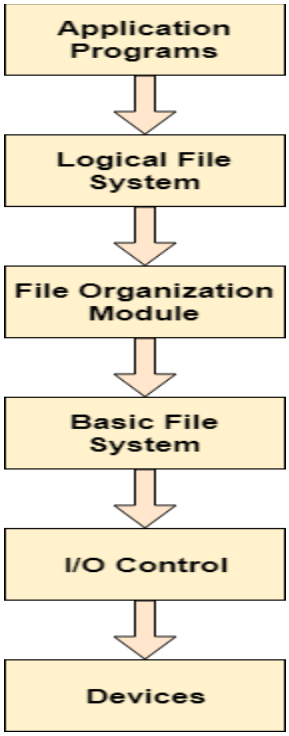
Every file has a set of permissions (called *access modes*) that determine who can read, modify, or execute the file.

- **File naming conventions**
  The name of each file must be unique within the directory where it is stored. This ensures that the file also has a unique path name in the file system.
- **File path names**
  The path name for each file and directory in the file system consists of the names of every directory that precedes it in the tree structure.
- **Pattern matching with wildcards and metacharacters**
  Wildcard characters provide a convenient way to specify multiple file names or directory names.
- **Pattern matching versus regular expressions**
  Regular expressions allow you to select specific strings from a set of character strings. The use of regular expressions is generally associated with text processing.

File System Structure

File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way. A file System must be able to store the file, locate the file and retrieve the file.

Most of the Operating Systems use layering approach for every task including file systems. Every layer of the file system is responsible for some activities.



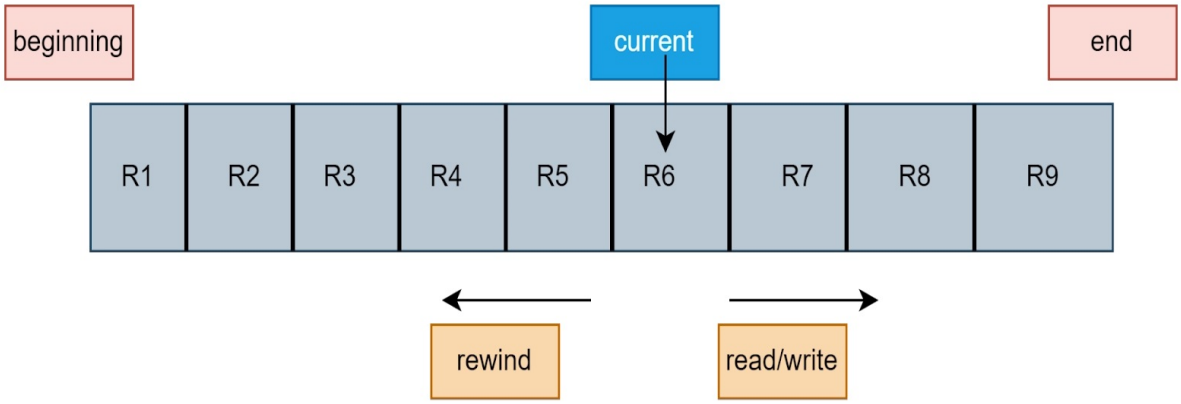
- When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure. If the application program doesn't have the required permissions of the file then this layer will throw an error. Logical file systems also verify the path to the file.
- Generally, files are divided into various logical blocks. Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors. Therefore, in order to store and retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.
- Once File organization module decided which physical block the application program needs, it passes this information to basic file system. The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.
- I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers. I/O controls are also responsible for handling interrupts.

File Access Methods in Operating System

When a file is used, information is read and accessed into computer memory and there are several ways to access this information of the file.

There are three ways to access a file into a computer system: Sequential-Access, Direct Access, Index sequential Method.

1. **Sequential Access –**
   It is the simplest access method. Information in the file is processed in order, one record after the other. This mode of access is by far the most common; for example, editor and compiler usually access the file in this fashion.
   Read and write make up the bulk of the operation on a file. A read operation -*read next*- read the next position of the file and automatically advance a file pointer, which keeps track I/O location. Similarly, for the -write *next*- append to the end of the file and advance to the newly written material.
   Key points:
   - Data is accessed one record right after another record in an order.
   - When we use read command, it move ahead pointer by one
   - When we use write command, it will allocate memory and move the pointer to the end of the file
   - Such a method is reasonable for tape.



Advantages of Sequential Access Method :

- It is simple to implement this file access mechanism.
- It uses lexicographic order to quickly access the next entry.
- It is suitable for applications that require access to all records in a file, in a specific order.
- It is less prone to data corruption as the data is written sequentially and not randomly.
- It is a more efficient method for reading large files, as it only reads the required data and does not waste time reading unnecessary data.
- It is a reliable method for backup and restore operations, as the data is stored sequentially and can be easily restored if required.

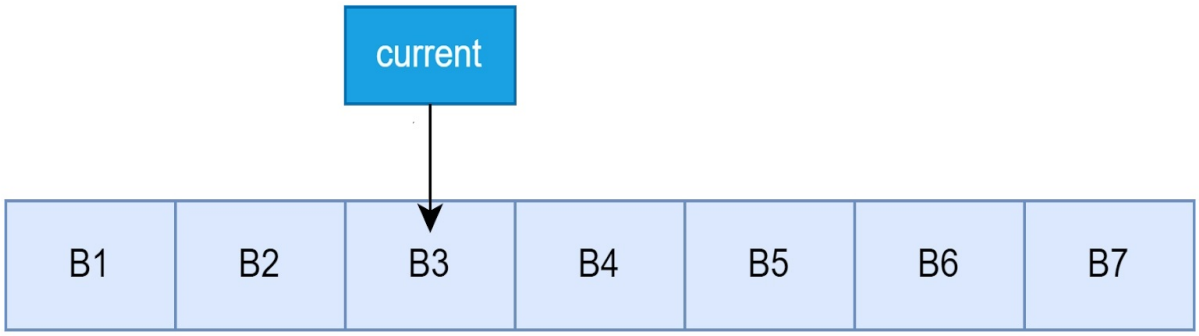
Disadvantages of Sequential Access Method :

- If the file record that needs to be accessed next is not present next to the current record, this type of file access method is slow.
- Moving a sizable chunk of the file may be necessary to insert a new record.
- It does not allow for quick access to specific records in the file. The entire file must be searched sequentially to find a specific record, which can be time-consuming.
- It is not well-suited for applications that require frequent updates or modifications to the file. Updating or inserting a record in the middle of a large file can be a slow and cumbersome process.
- Sequential access can also result in wasted storage space if records are of varying lengths. The space between records cannot be used by other records, which can result in inefficient use of storage.

2.Direct Access –

Another method is *direct access method* also known as *relative access method*. A fixed-length logical record that allows the program to read and write record rapidly. in no particular order. The direct access is based on the disk model of a file since disk allows random access to any file block. For direct access, the file is viewed as a numbered sequence of block or record. Thus, we may read block 14 then block 59, and then we can write block 17. There is no restriction on the order of reading and writing for a direct access file.

A block number provided by the user to the operating system is normally a *relative block number*, the first relative block of the file is 0 and then 1 and so on.



- **Advantages of the Direct Access Method**
- Some advantages of the Direct Access Method include the following:
- Random access to specific data within a file allows for quick and efficient data retrieval.
- High concurrency, enabling multiple processes to access the same file simultaneously and efficiently share data.
- Flexible data retrieval, suitable for accessing data in a non-sequential fashion.
- Efficient data modification, as changes to specific data can be made without rewriting the entire file.
- Suitable for real-time systems that require fast and predictable data access.

- **Disadvantages of the Direct Access Method**
- Some disadvantages of the Direct Access Method include the following:
- **Inefficiency for Small Requests:** The Direct Access Method can be inefficient when you need to retrieve small pieces of data frequently. This is because the method is optimized for accessing larger chunks of data, so it might not work efficiently for tiny bits of information.

- **Space Management Challenges**: Managing the available space using the Direct Access Method can be complex. It's like arranging things in a box – if you're not careful, things can get disorganized and jumbled up. This can lead to wasted space and difficulties in keeping track of where data is stored.

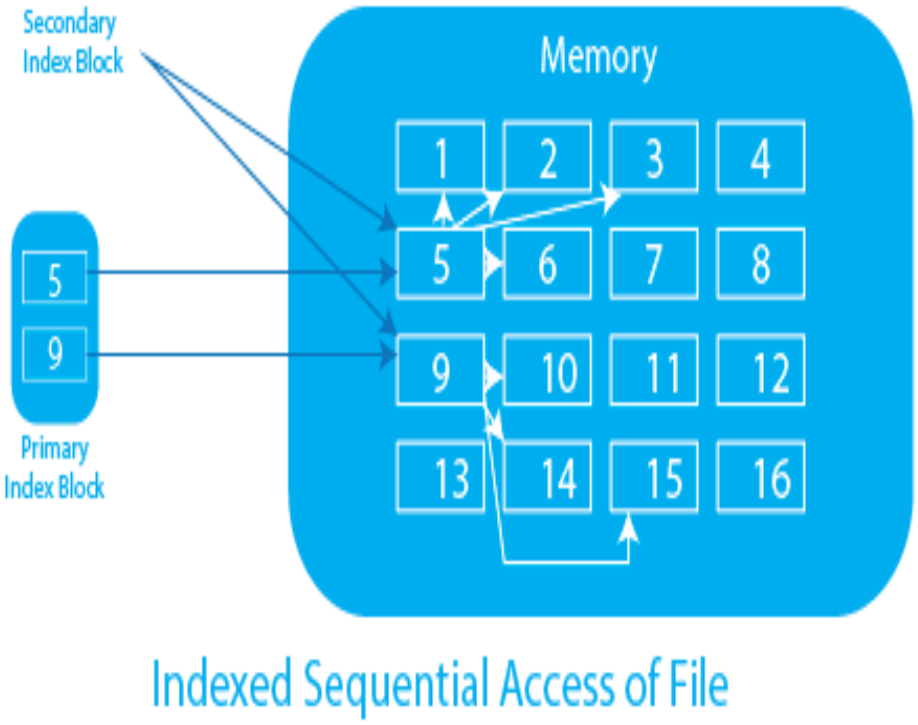
- **Fragmentation Issues:** Fragmentation can happen when there are gaps between stored data blocks. These gaps can result from adding, updating, or deleting records. Like a

jigsaw puzzle with missing pieces, fragmentation makes it less efficient to use the available space.

- **Varied Record Lengths:** The Direct Access Method might not work well when dealing with records of different lengths. Imagine trying to fit objects of different sizes into fixed compartments – it can be challenging and lead to inefficiencies in storage and retrieval.

- **Suboptimal Performance Patterns:** Certain data access patterns might not work optimally with the Direct Access Method. It's like taking different routes to your destination – sometimes, the method might not be the fastest or most efficient way to retrieve the data you need.

3.Index sequential method –
It is the other method of accessing a file that is built on the top of the sequential access method. These methods construct an index for the file. The index, like an index in the back of a book, contains the pointer to the various blocks. To find a record in the file, we first search the index, and then by the help of pointer we access the file directly.
Key points:
- It is built on top of Sequential access.
- It control the pointer by using index.



Indexed Sequential Access of File

- **Advantages of Indexed Access**
  Some of the advantages of indexed access are given below:

- It provides fast access to all the data blocks within the file as we can directly locate the data block easily.

- It is very efficient for large files as it saves a lot of searching time.

- It has better memory management as it allocation space to the index and data blocks separately. And this will prevent memory overflow.

- It will improve data integrity by maintaining a separate index for each file.

- **Disadvantages of Indexed Access**
  We have explained some of the disadvantages of indexed access below:

- There can be index overhead with creating and maintaining the index. As we have to update the index every time a record is modified or updated.

- The storage requirements increase as the additional storage space to store the index

- The concurrent access is limited as multiple users cannot modify the same index simultaneously.

- The complexity will increase due to the addition of an index in the file management system.

- **4.Relative Record Access –**
- Relative record access is a file access method used in operating systems where records are accessed relative to the current position of the file pointer. In this method, records are located based on their position relative to the current record, rather than by a specific address or key value.

- **Key Points of Relative Record Access:**
- Relative record access is a random access method that allows records to be accessed based on their position relative to the current record.

- This method is efficient for accessing individual records but may not be suitable for files that require frequent updates or random access to specific records.

- Relative record access requires fixed-length records and may not be flexible enough for some applications.

- This method is useful for processing records in a specific order or for files that are accessed sequentially.

- **Advantages of Relative Record Access:**

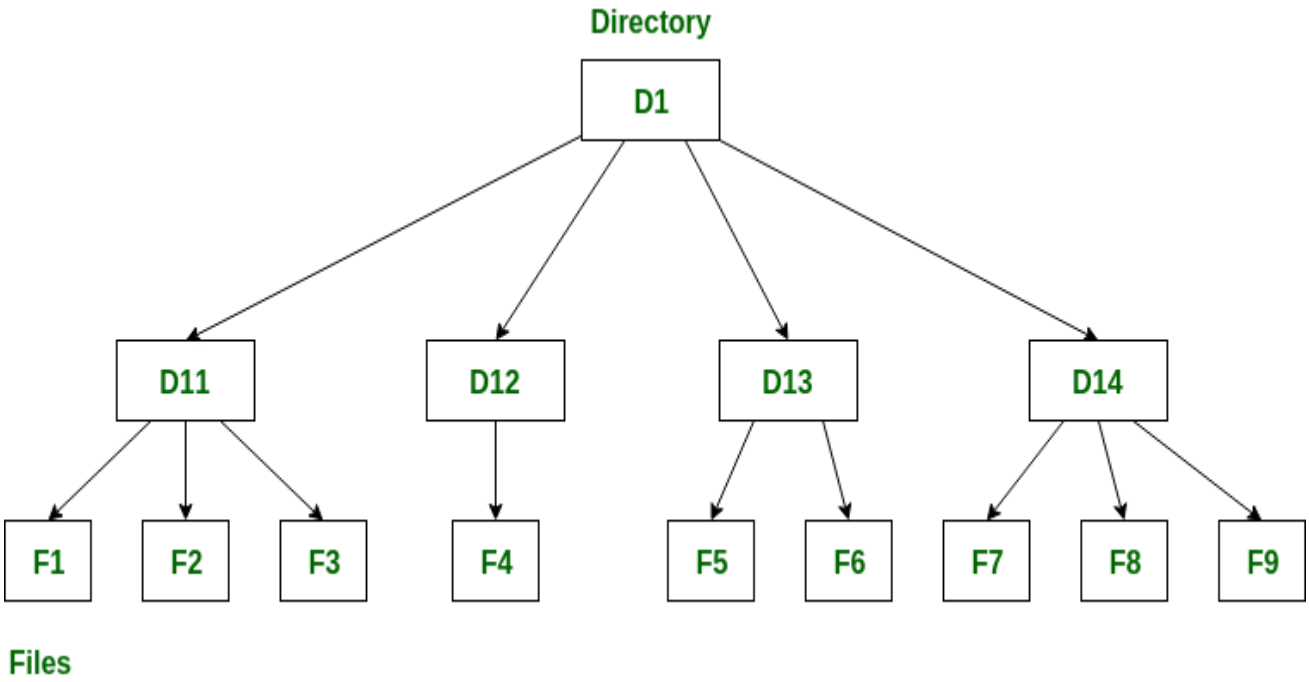
- Random Access: Relative record access allows random access to records in a file. The system can access any record at a specific offset from the current position of the file pointer.
- Efficient Retrieval: Since the system only needs to read the current record and any records that need to be skipped, relative record access is more efficient than sequential access for accessing individual records.
- Useful for Sequential Processing: Relative record access is useful for processing records in a specific order. For example, if the records are sorted in a specific order, the system can access the next or previous record relative to the current position of the file pointer.
- **Disadvantages of Relative Record Access:**
- Fixed Record Length: Relative record access requires fixed-length records. If the records are of varying length, it may be necessary to use padding to ensure that each record is the same length.
- Limited Flexibility: Relative record access is not very flexible. It is difficult to insert or delete records in the middle of a file without disrupting the relative positions of other records.
- Limited Application: Relative record access is best suited for files that are accessed sequentially or with some regularity, but it may not be appropriate for files that are frequently updated or require random access to specific records.
- **5.Content Addressable Access-**
- Content-addressable access (CAA) is a file access method used in operating systems that allows records or blocks to be accessed based on their content rather than their address. In this method, a hash function is used to calculate a unique key for each record or block, and the system can access any record or block by specifying its key.
- **Keys in Content-Addressable Access:**
- Unique: Each record or block has a unique key that is generated using a hash function.
- Calculated based on content: The key is calculated based on the content of the record or block, rather than its location or address.
- **Advantages of Content-Addressable Access:**
- Efficient Search: CAA is ideal for searching large databases or file systems because it allows for efficient searching based on the content of the records or blocks.
- Flexibility: CAA is more flexible than other access methods because it allows for easy insertion and deletion of records or blocks.
- Data Integrity: CAA ensures data integrity because each record or block has a unique key that is generated based on its content.
- **Disadvantages of Content-Addressable Access:**
- Overhead: CAA requires additional overhead because the hash function must be calculated for each record or block.

- Collision: There is a possibility of collision where two records or blocks can have the same key. This can be minimized by using a good hash function, but it cannot be completely eliminated.
- Limited Key Space: The key space is limited by the size of the hash function used, which can lead to collisions and other issues.
- **Key Points of Content-Addressable Access:**
- Content-addressable access is a file access method that allows records or blocks to be accessed based on their content rather than their address.
- CAA uses a hash function to generate a unique key for each record or block.
- CAA is efficient for searching large databases or file systems and is more flexible than other access methods.
- CAA requires additional overhead for calculating the hash function and may have collisions or limited key space.

Directory and Disk Structure

Directory Structure

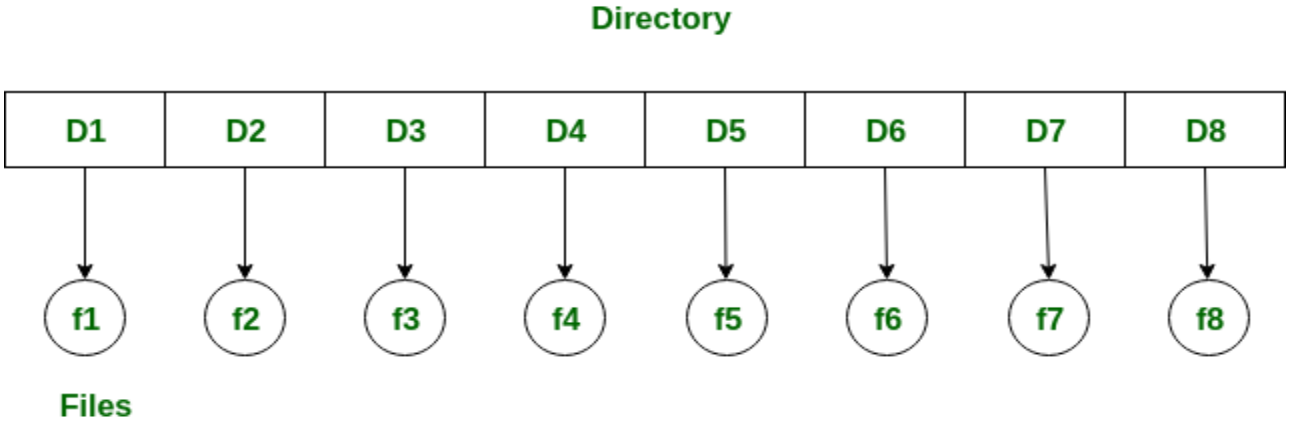
A **directory** is a container that is used to contain folders and files. It organizes files and folders in a hierarchical manner.



Following are the logical structures of a directory, each providing a solution to the problem faced in previous type of directory structure.

1) <u>Single-level directory:</u>

The single-level directory is the **simplest directory structure**. In it, all files are contained in the same directory which makes it easy to support and understand.
A single level directory has a significant limitation, however, when the number of files increases or when the system has more than one user. Since all the files are in the same directory, they must have a **unique name**. If two users call their dataset test, then the unique name rule violated.



Advantages:

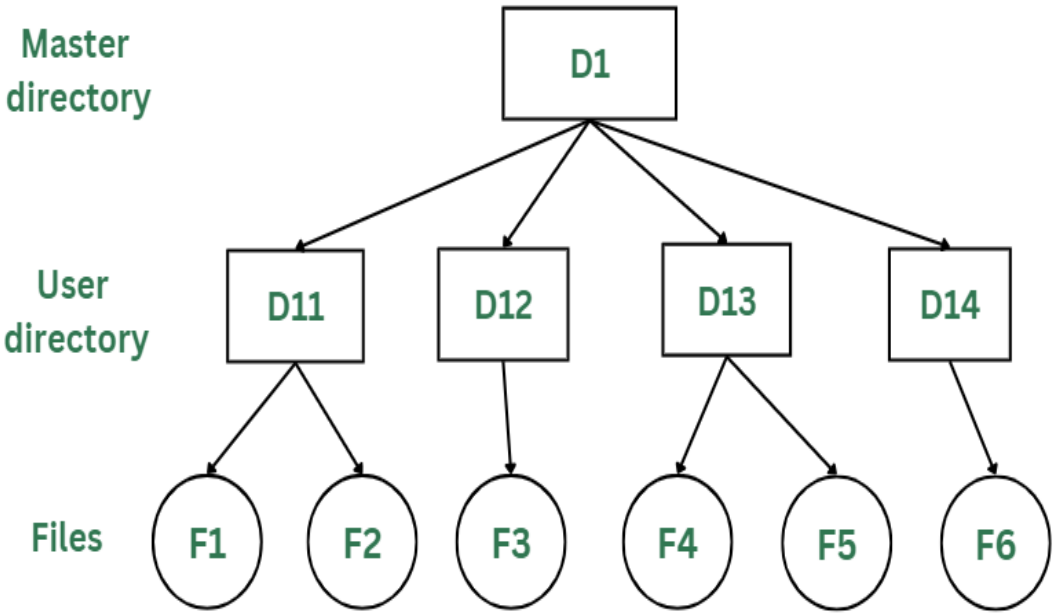
- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating are very easy in such a directory structure.
- **Logical Organization**: Directory structures help to logically organize files and directories in a hierarchical structure. This provides an easy way to navigate and manage files, making it easier for users to access the data they need.
- **Increased Efficiency:** Directory structures can increase the efficiency of the file system by reducing the time required to search for files. This is because directory structures are optimized for fast file access, allowing users to quickly locate the file they need.
- **Improved Security**: Directory structures can provide better security for files by allowing access to be restricted at the directory level. This helps to prevent unauthorized access to sensitive data and ensures that important files are protected.
- **Facilitates Backup and Recovery**: Directory structures make it easier to backup and recovery files in the event of a system failure or data loss. By storing related files in the same directory, it is easier to locate and backup all the files that need to be protected.
- **Scalability:** Directory structures are scalable, making it easy to add new directories and files as needed. This helps to accommodate growth in the system and makes it easier to manage large amounts of data.

Disadvantages:

- There may change of name collision because two files can have the same name.
- Searching will become time taking if the directory is large.
- This can not group the same type of files together.

2) <u>Two-level directory:</u>

As we have seen, a single level directory often leads to confusion of files names among different users. The solution to this problem is to create a **separate directory for each user**. In the two-level directory structure, each user has their own **user files directory (UFD).** The UFDs have similar structures, but each lists only the files of a single user. System's **master file directory (MFD*)*** is searched whenever a new user id is created.



Two-Levels Directory Structure

Advantages:

- The main advantage is there can be more than two files with same name, and would be very helpful if there are multiple users.
- A security would be there which would prevent user to access other user's files.
- Searching of the files becomes very easy in this directory structure.

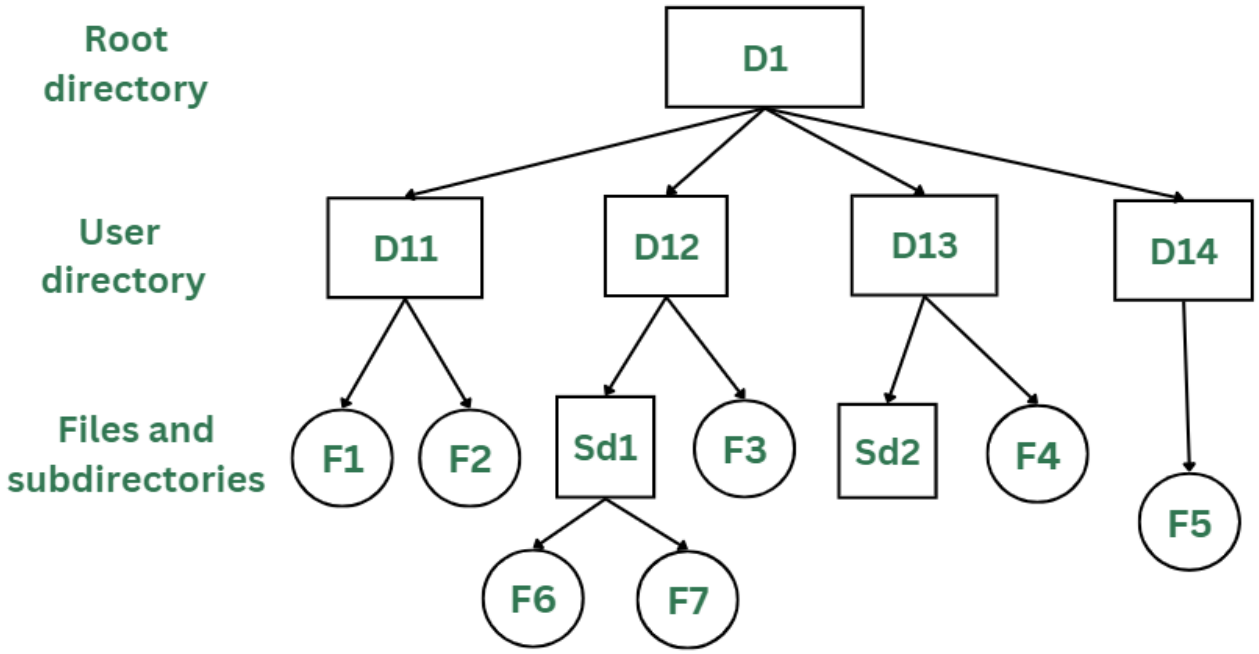
Disadvantages:

- As there is advantage of security, there is also disadvantage that the user cannot share the file with the other users.
- Unlike the advantage users can create their own files, users don't have the ability to create subdirectories.
- Scalability is not possible because one use can't group the same types of files together.

3) Tree Structure/ Hierarchical Structure:

Tree directory structure of operating system is most commonly used in our **personal computers**. User can create files and subdirectories too, which was a disadvantage in the previous directory structures.

This directory structure resembles a real tree upside down, where the **root directory** is at the peak. This root contains all the directories for each user. The users can create subdirectories and even store files in their directory.
A user do not have access to the root directory data and cannot modify it. And, even in this directory the user do not have access to other user's directories. The structure of tree directory is given below which shows how there are files and subdirectories in each user's directory.



Tree/Hierarchical Directory Structure

Advantages:

- This directory structure allows subdirectories inside a directory.
- The searching is easier.
- File sorting of important and unimportant becomes easier.
- This directory is more scalable than the other two directory structures explained.

Disadvantages:

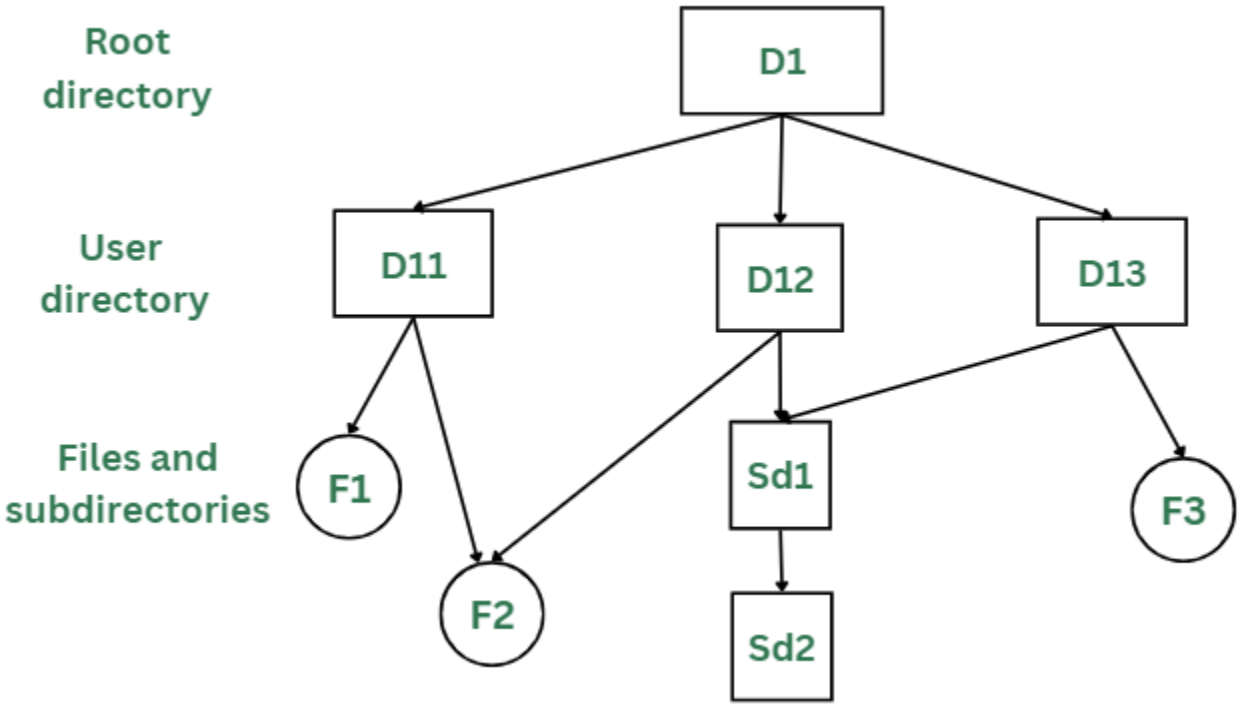
- As the user isn't allowed to access other user's directory, this prevents the file sharing among users.
- As the user has the capability to make subdirectories, if the number of subdirectories increase the searching may become complicated.
- Users cannot modify the root directory data.
- If files do not fit in one, they might have to be fit into other directories.

4) Acyclic Graph Structure:

As we have seen the above three directory structures, where none of them have the capability to access one file from multiple directories. The file or the subdirectory could be accessed through the directory it was present in, but not from the other directory.

This problem is solved in acyclic graph directory structure, where a file in one directory can be accessed from multiple directories. In this way, the files could be shared in between the users. It is designed in a way that multiple directories point to a particular directory or file with the help of links.

In the below figure, this explanation can be nicely observed, where a file is shared between multiple users. If any user makes a change, it would be reflected to both the users.



Acyclic Graph Structure

Advantages:

- Sharing of files and directories is allowed between multiple users.
- Searching becomes too easy.
- Flexibility is increased as file sharing and editing access is there for multiple users.

Disadvantages:

- Because of the complex structure it has, it is difficult to implement this directory structure.
- The user must be very cautious to edit or even deletion of file as the file is accessed by multiple users.
- If we need to delete the file, then we need to delete all the references of the file inorder to delete it permanently.

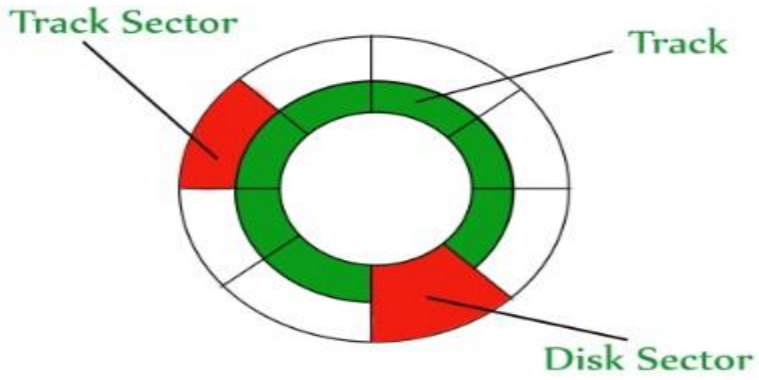
Disk Structure in OS

Hard Disk is a secondary storage device. It is a type of **electro mechanical device**. A hard disk stores and retrieves digital data using magnetic storage. Disk is rigid rapidly rotating platters coated with magnetic material.
Hard Disk Pack consist of more than one disk.

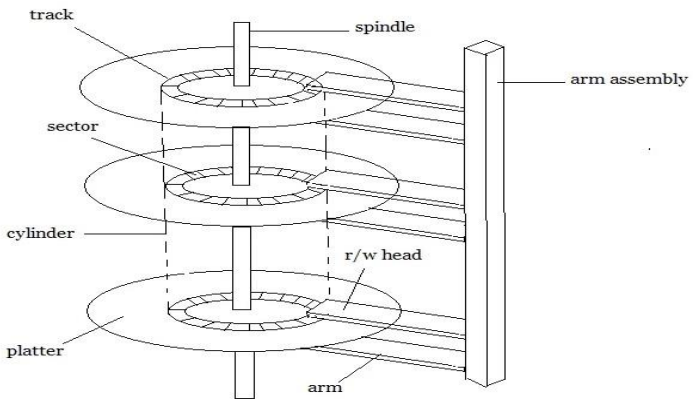
How Data is Stored in a Hard Disk ?

- Data is stored in Disk in form of tracks and sectors.
- Disk surface is divided in to tracks.
- Track is further dived in to sectors.
- Sector is the addressable unit in disk.

Basic picture for disk storage concept is given below



Disk structure is as shown in following figure



Disk Pack Structure

Step wise description of Disk Structure is given below

- Disk surface divided into tracks
- A read/write head positioned just above the disk surface
- Information stored by magnetic recording on the track under read/write head
- Fixed head disk
- Moving head disk
- Designed for large amount of storage
- Primary design consideration cost, size, and speed

Hardware for disk system

- Disk drive, Device motor, Read/write head, Associated logic

Disk controller

- Determines the logical interaction with the computer
- Can service more than one drive (overlapped seeks)

Cylinder

- The same numbered tracks on all the disk surfaces
- Each track contains between 8 to 32 sectors

Sector

- Smallest unit of information that can be read from/written into disk
- Range from 32 bytes to 4096 bytes

Disk Performance Parameters

Some important parameters used to measure the performance of hard disk are as follow

Seek time

- Seek Time is time required by read/write head to move to requested track
- Includes the initial startup time and the time required to traverse the tracks to be crossed once the access arm is up to speed.

Latency or rotational delay

- Time required for the requested sector to come under the read/write head.
- Rotational delay is generally the half of the time taken in one rotation.

Data Transfer Rate

Data transfer rate is define as the amount of data transfer in per unit time for example 30 MB/Sec.

Data Transfer Time

Data Transfer time is the total time taken to transfer a specific amount of data from the disk. Data Transfer time depends on the data transfer rate of the disk.

Average Access Time

Average access time is calculated as

Average Access Time = Seek Time + Rotational Latecny + Data Transfer Time

File System Implementation

Partitioning

Fixed Partitioning

The earliest and one of the simplest technique which can be used to load more than one processes into the main memory is Fixed partitioning or Contiguous memory allocation.

In this technique, the main memory is divided into partitions of equal or different sizes. The operating system always resides in the first partition while the other partitions can be used to store user processes. The memory is assigned to the processes in contiguous way.

In fixed partitioning,

1. The partitions cannot overlap.
2. A process must be contiguously present in a partition for the execution.

There are various cons of using this technique.

1. Internal Fragmentation

If the size of the process is lesser then the total size of the partition then some size of the partition get wasted and remain unused. This is wastage of the memory and called internal fragmentation.

As shown in the image below, the 4 MB partition is used to load only 3 MB process and the remaining 1 MB got wasted.

2. External Fragmentation

The total unused space of various partitions cannot be used to load the processes even though there is space available but not in the contiguous form.

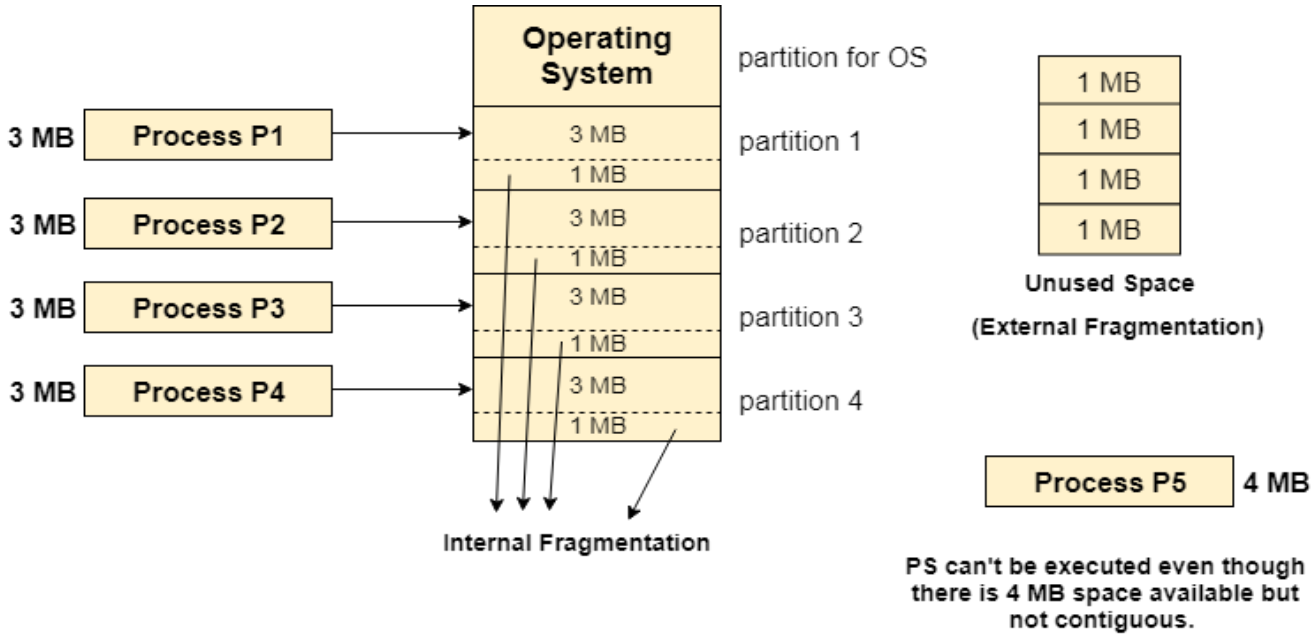
As shown in the image below, the remaining 1 MB space of each partition cannot be used as a unit to store a 4 MB process. Despite of the fact that the sufficient space is available to load the process, process will not be loaded.

3. Limitation on the size of the process

If the process size is larger than the size of maximum sized partition then that process cannot be loaded into the memory. Therefore, a limitation can be imposed on the process size that is it cannot be larger than the size of the largest partition.

4. Degree of multiprogramming is less

By Degree of multi programming, we simply mean the maximum number of processes that can be loaded into the memory at the same time. In fixed partitioning, the degree of multiprogramming is fixed and very less due to the fact that the size of the partition cannot be varied according to the size of processes.



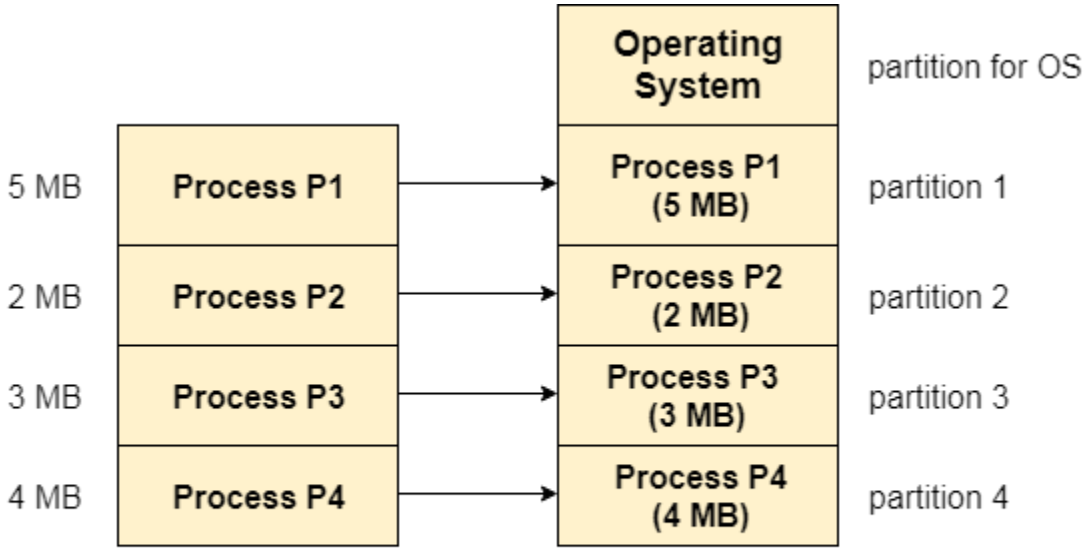
Fixed Partitioning

(Contiguous memory allocation)

Dynamic Partitioning

Dynamic partitioning tries to overcome the problems caused by fixed partitioning. In this technique, the partition size is not declared initially. It is declared at the time of process loading.

The first partition is reserved for the operating system. The remaining space is divided into parts. The size of each partition will be equal to the size of the process. The partition size varies according to the need of the process so that the internal fragmentation can be avoided.



Dynamic Partitioning

(Process Size = Partition Size)

Advantages of Dynamic Partitioning over fixed partitioning

1. No Internal Fragmentation

Given the fact that the partitions in dynamic partitioning are created according to the need of the process, It is clear that there will not be any internal fragmentation because there will not be any unused remaining space in the partition.

2. No Limitation on the size of the process

In Fixed partitioning, the process with the size greater than the size of the largest partition could not be executed due to the lack of sufficient contiguous memory. Here, In Dynamic partitioning, the process size can't be restricted since the partition size is decided according to the process size.

3. Degree of multiprogramming is dynamic

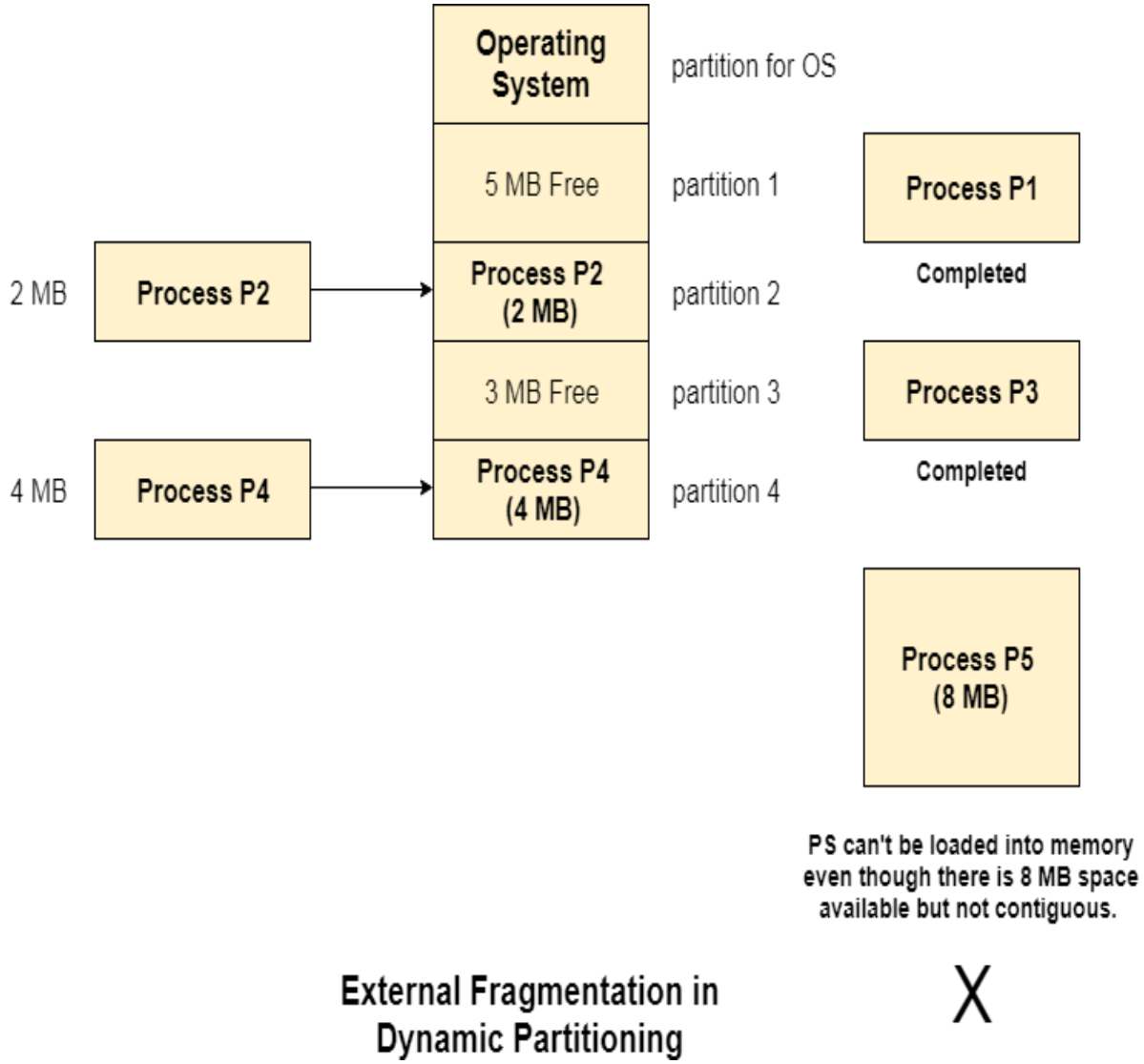
Due to the absence of internal fragmentation, there will not be any unused space in the partition hence more processes can be loaded in the memory at the same time.

Disadvantages of dynamic partitioning

External Fragmentation

Absence of internal fragmentation doesn't mean that there will not be external fragmentation.

Let's consider three processes P1 (1 MB) and P2 (3 MB) and P3 (1 MB) are being loaded in the respective partitions of the main memory.After some time P1 and P3 got completed and their assigned space is freed. Now there are two unused partitions (1 MB and 1 MB) available in the main memory but they cannot be used to load a 2 MB process in the memory since they are not contiguously located.The rule says that the process must be contiguously present in the main memory to get executed. We need to change this rule to avoid external fragmentation.



External Fragmentation in Dynamic Partitioning

Complex Memory Allocation

In Fixed partitioning, the list of partitions is made once and will never change but in dynamic partitioning, the allocation and deallocation is very complex since the partition size will be varied every time when it is assigned to a new process. OS has to keep track of all the partitions.

Due to the fact that the allocation and deallocation are done very frequently in dynamic memory allocation and the partition size will be changed at each time, it is going to be very difficult for OS to manage everything.

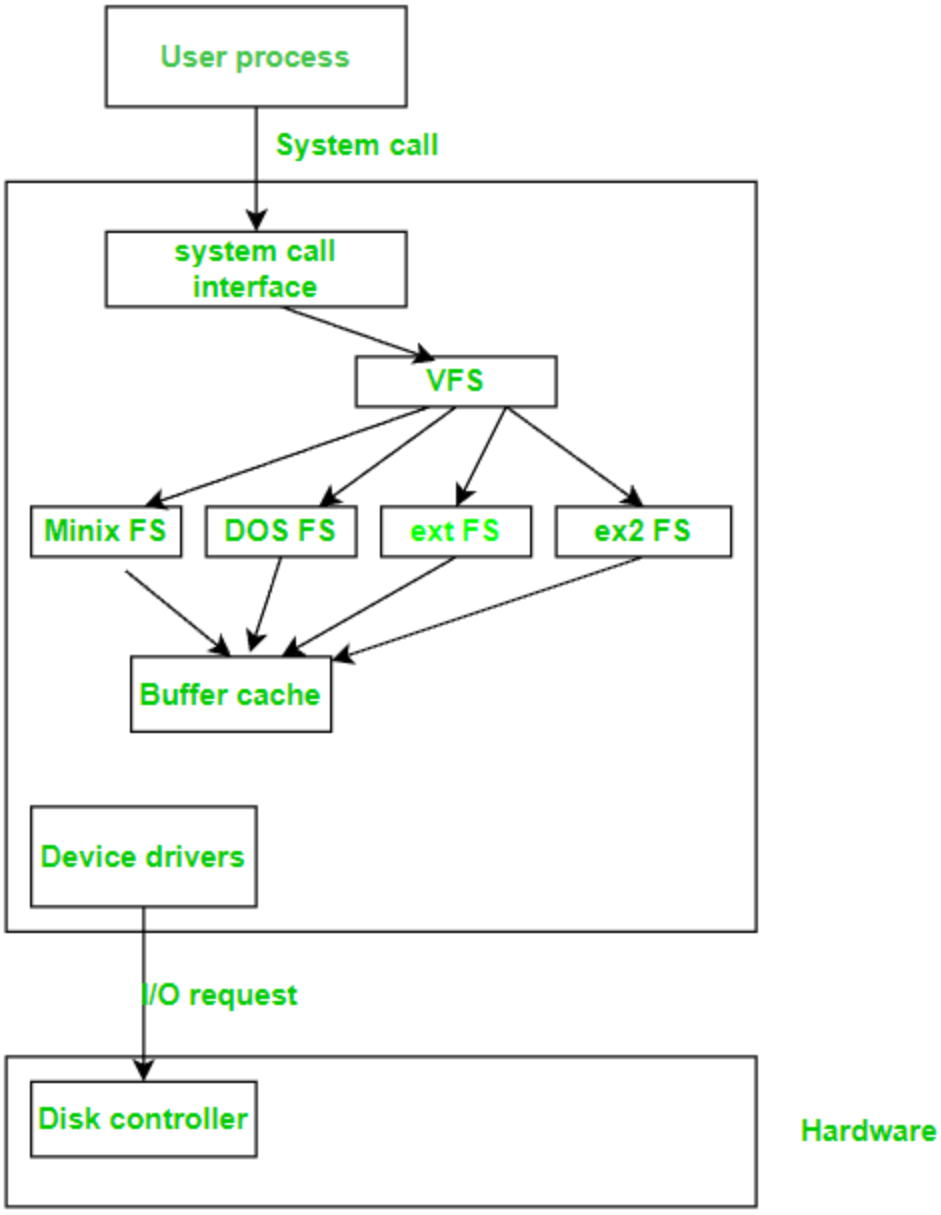
Virtual File System

The virtual file system is one of the best features of Linux that makes Linux an operating system of choice. Format a pen drive in any Linux flavor operating system and try to see the contents in windows OS. Windows OS does not show anything. What is the problem?

Now try doing the same in windows i.e format a Pendrive in the windows operating system to store some files and then try to view these files through any Linux operating system. we will be able to see the contents. we will also be able to open and read file content easily.

How is that possible? The answer to these questions lies in the concept of the virtual file system that Linux support. VFX is a file system interface for a user process. VFS has the capability of conceiving features and behavior of any file system regardless of processor type memory format as well as hardware.

Two mapping functions are used that transform the properties of the real-time system to the characteristics required by VFS.



Virtual File System

As shown in the diagram:
The user process initiates the execution of the system call with the help of the VFS method. VFS interprets this stimulus into an appropriate internal file system call which in turn activates the concerned mapping function of the specified file system.

In the case of an operating system that demands structural changes, all required constructs are created dynamically to meet the requirements. Ex: In FAT-compatible systems, directories are

not treated as files so such characteristics are dynamically created since they cannot be mapped. The target file system is then activated to take control so that results can be delivered to the user of the system.

As seen in the above diagram **actions** and **reactions** are observed as a role of VFS:
The user initiates a file-oriented system call.

1. Kernel calls function in VFS.
2. Execution of file system independent manipulation and initiation of target file system call.
3. mapping of calls from VFS to the target file system.
4. The target file system converts the request into device driver execution.
5. Appropriate action is completed as per the guidance of the device driver function and the results are obtained.
6. Since VFS objects are implemented as C data structures. Every object is attached with data and pointers.

Primary Types in VFS

1. **Superblock object:** point out specific mount points.
2. **inode object**: Relates specific file.
3. **Directory:** Shows entry to a specific directory.
4. **File object:** Depicts an open file that is used by the process.

1. Tmpfs virtual file system: A virtual file system for Linux called tmpfs stores data in the system's virtual memory. The internal caches of the kernel serve as temporary storage for all files, much like any other virtual file system.

2. Sysfs virtual file system: When a device is added or withdrawn, the sysfs virtual file system dynamically updates the user-specified view of the system's devices. The global, bus, and device discovery levels are represented by the directory hierarchy beneath /sys. The files on the global layer record the device's power state, which is also programmable. The files in the bus layer report details about each device's resources and IRQ number. The file system contains files that a device driver can use to access data or configurable interfaces.

The filesystem type, which is specified in **include/linux/fs.h** (and abbreviated here for brevity), must be known by the kernel in order to use a filesystem:

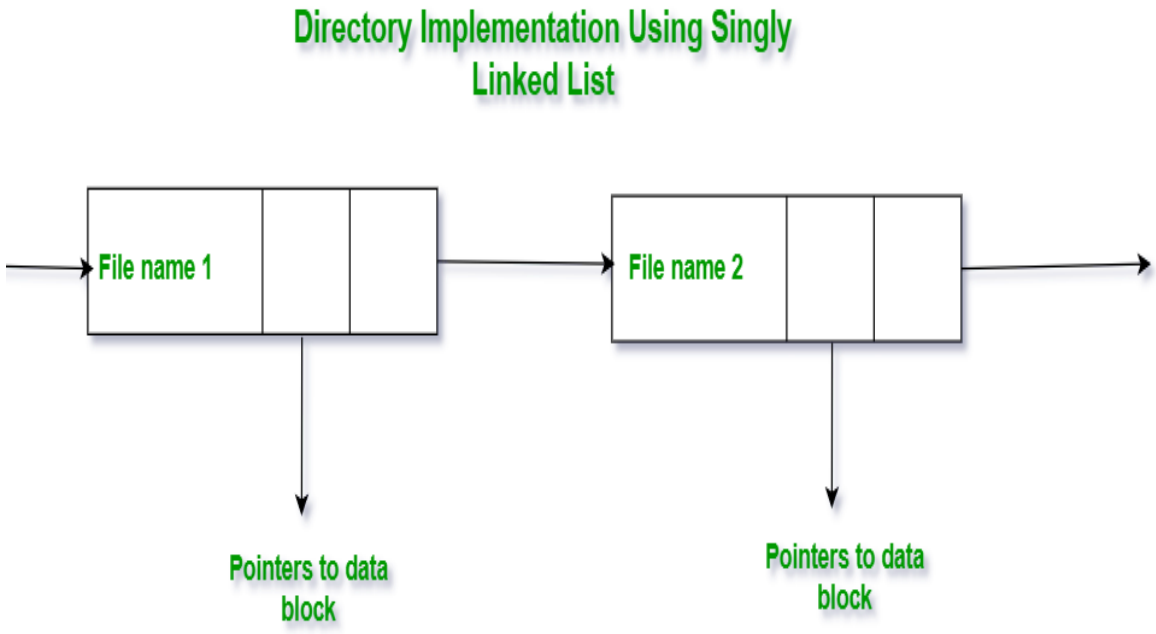
- **struct file_system_type** maintains data related to a filesystem but is not associated with any particular '**instance of a filesystem'.**
- The userspace command **cat /proc/filesystems** can be used to inspect the linked list of known filesystem types that the VFS keeps.
- Filesystem types are unregistered and registered in the kernel through these functions in **include/linux/fs.h:**
- A filesystem of a certain type may be mounted to a place in the directory tree if the kernel is aware of it (i.e., it has been registered).

Directory Implementation in Operating System

Directory implementation in the operating system can be done using Singly Linked List and Hash table. The efficiency, reliability, and performance of a file system are greatly affected by the selection of directory-allocation and directory-management algorithms. There are numerous ways in which the directories can be implemented. But we need to choose an appropriate directory implementation algorithm that enhances the performance of the system.

Directory Implementation using Singly Linked List

The implementation of directories using a singly linked list is easy to program but is time-consuming to execute. Here we implement a directory by using a linear list of filenames with pointers to the data blocks.



Directory Implementation Using Singly Linked List

- To create a new file the entire list has to be checked such that the new directory does not exist previously.
- The new directory then can be added to the end of the list or at the beginning of the list.
- In order to delete a file, we first search the directory with the name of the file to be deleted. After searching we can delete that file by releasing the space allocated to it.
- To reuse the directory entry we can mark that entry as unused or we can append it to the list of free directories.
- To delete a file linked list is the best choice as it takes less time.

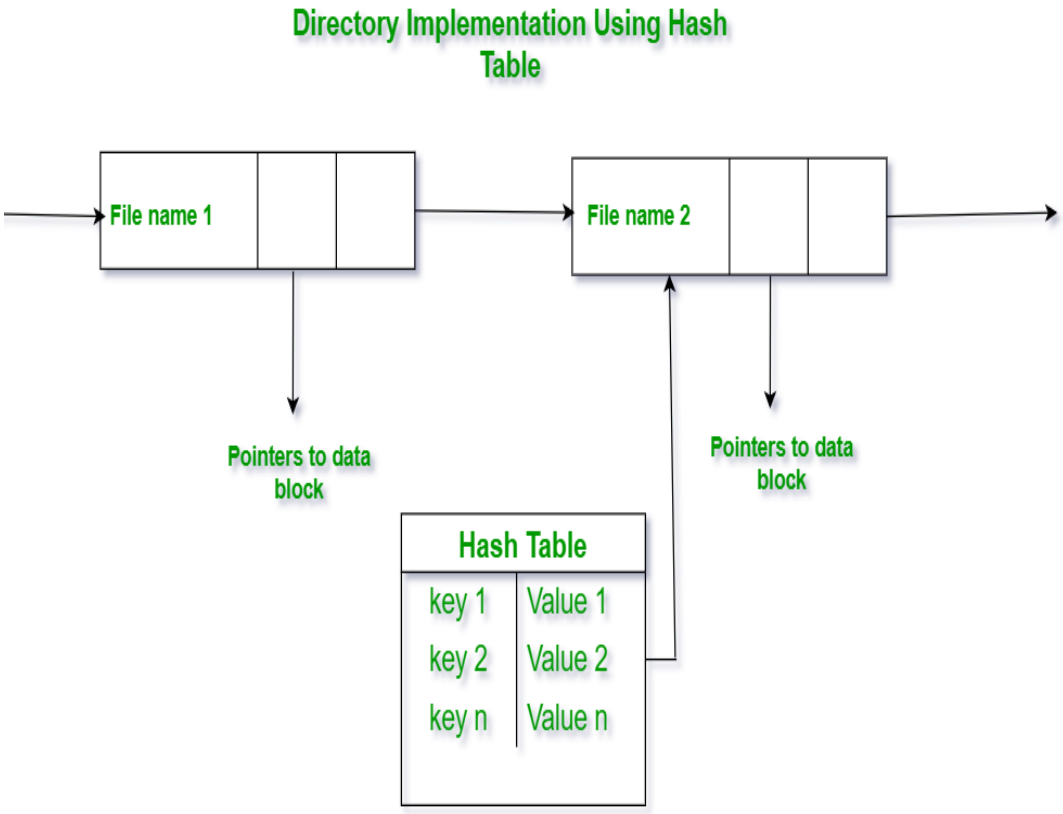
Disadvantage

The main disadvantage of using a linked list is that when the user needs to find a file the user has to do a linear search. In today's world directory information is used quite frequently and linked list implementation results in slow access to a file. So the operating system maintains a cache to store the most recently used directory information.

Directory Implementation using Hash Table

An alternative data structure that can be used for directory implementation is a hash table. It overcomes the major drawbacks of directory implementation using a linked list. In this method, we use a hash table along with the linked list. Here the linked list stores the directory entries, but a hash data structure is used in combination with the linked list.

In the hash table for each pair in the directory key-value pair is generated. The hash function on the file name determines the key and this key points to the corresponding file stored in the directory. This method efficiently decreases the directory search time as the entire list will not be searched on every operation. Using the keys the hash table entries are checked and when the file is found it is fetched.



Directory Implementation Using Hash Table

Disadvantage:

The major drawback of using the hash table is that generally, it has a fixed size and its dependency on size. But this method is usually faster than linear search through an entire directory using a linked list.

File System Mounting in OS

Mounting is a process in which the operating system adds the directories and files from a storage device to the user's computer file system. The file system is attached to an empty directory, by adding so the system user can access the data that is available inside the storage device through the system file manager. Storage systems can be internal hard disks, external hard disks, USB flash drivers, SSD cards, memory cards, network-attached storage devices, CDs and DVDs, remote file systems, or anything else.

Terminologies used in File System Mounting

- **File System:** It is the method used by the operating system to manage data storage in a storage device. So, a user can access and organize the directories and files in an efficient manner.
- **Device name:** It is a name/identifier given to a storage partition. In windows, for example, "D:" in windows.
- **Mount point**: It is an empty directory in which we are adding the file system during the process of mounting.

Mounting Indifferent Operating Systems

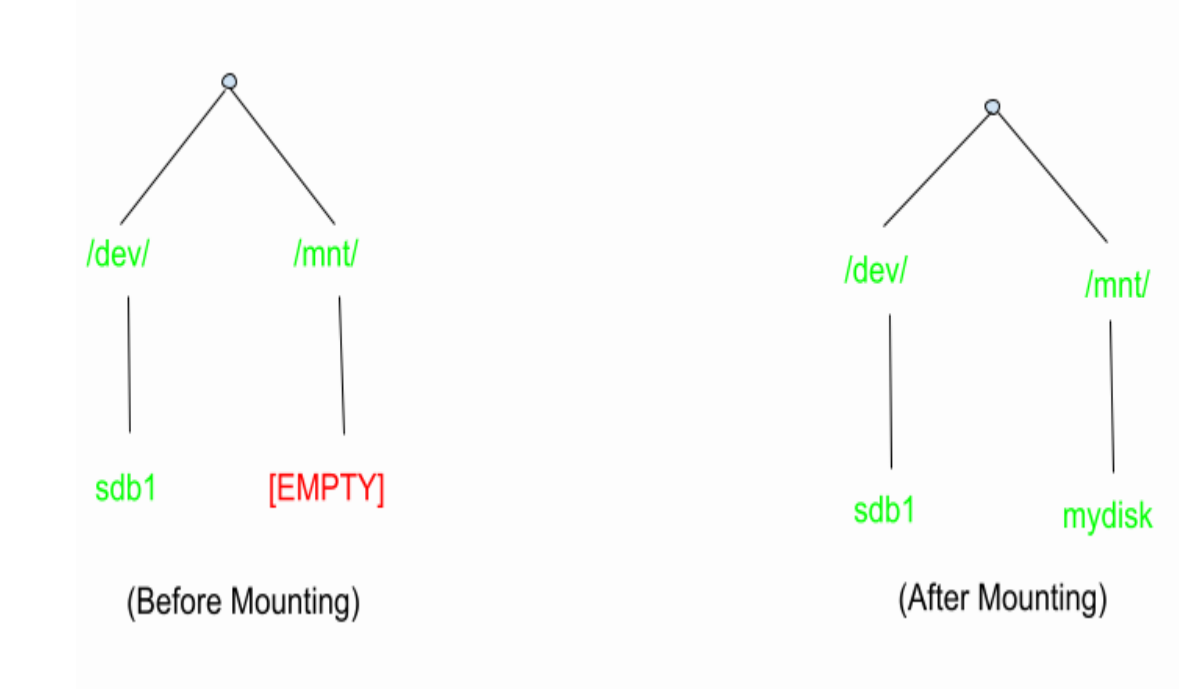
1. Linux-Unix based OS

We want to mount ***/dev/sdb1*** to an existing directory ***/mnt***.

sudo mount /dev/sdb1 /mnt/mydisk

After mounting, we have to unmount after use

sudo umount /mnt/mydisk



before and after mounting

2. Windows OS

In windows mounting is very easy for a user. When we connect the external storage devices, windows automatically detect the file system and mount it to the drive letter. Drive letter may be **D:** or **E:**.

Steps:

- Connect an external storage device to your PC.
- Windows detects the file system on the drive (e.g., FAT32 or NTFS) and assigns it a drive letter, such as “E:”.
- You can access the derive by going through, THIS PC –> FILE EXPLORER –>”E:” drive
- Access the data.

3. Mac OS

In Mac OS when we connect an external storage it will automatically mount, and it will be accessible via Finder. As an advanced mounting method user can also use the command **diskutil** in Terminal.

Method 1:

Steps:

- Connect an external storage device to your MAC.
- MS OS detects the file system and automatically mount it.
- You can access the drive by opening Finder, and it will appear in the sidebar.

Method 2(Using diskutil):

To mount a drive with a known identifier: ***disk2s1***
diskutil mount /dev/disk2s1
To unmount:
diskutil unmount /dev/disk2s1

Disc Management in OS?

Disk management in an operating system is the process of organizing and managing physical storage devices, like solid-state drives (SSDs) and hard disk drives, in a computer system. It includes creating, formatting, editing, partitioning, and managing disk volumes to store and access data efficiently. The operating system keeps track of the files and space on the physical drives and manages the associated read and write operations.

Disk Management tasks like managing the hard drives and storage are performed in the Windows Operating System using a graphical interface. We can create, format, and resize different parts of our disks and assign letters or rename them for easy access.

In contrast, the Linux Operating System doesn't have a graphical tool like Windows. It has a command-line program called "fdisk." which allows users to perform similar things like creating, deleting, and managing disk partitions.

The operating system's disc management includes:

- Disk Format (or low-level format or physical format)
- Booting from disk

- Bad block recovery

Now, we will learn about them in detail.

Also see, Multiprogramming vs Multitasking And Open Source Operating System

Disk Format

Before storing data, it divides the disc into sectors so that the disc controller can read and write. Each section might be anything from:

Optional discs employ the operating system's own data structures to maintain files utilizing discs, but the header retains information, data, and error correction code (ECC) sectors of data, typically 512 bytes of data.

It is carried out in two stages:

1. Separate the disc into cylinder groups. Each one is considered a logical disc.
2. Create a file system in a logical format. On the disc, the OS saves the data structure of the first file system. It has both free and assigned space.

Most file systems organize blocks into clusters for efficiency. Disk I/O happens in chunks. File I/O is performed in a cluster.

Now, we will learn about Booting from disk, this is also known as boot block.

Boot block

A system must run an initial program when it is turned on or restarted. The bootstrap program is the system's initialization program. It begins the OS after all of the system's components have been initialized. The bootstrap program searches for the OS kernel on the disc, load it into memory, and then jumps to a beginning address to begin OS operation.

On most computer systems, the bootstrap is kept in read-only memory. It's useful since read-only memory doesn't need to be initialized and is in a fixed location where the CPU may start executing regardless of whether the computer is turned on or off. Furthermore, because ROM is read-only, it may not be impacted by a computer virus. The problem is that upgrading the ROM hardware chips are required to update this bootstrap code.

As a result, most computer systems include minimal boot ROM software whose primary role is to load a full bootstrap program from a disc drive. The entire bootstrap program is easily customizable, and the disc is rewritten with a new version. The boot block contains the bootstrap program stored in a partition. A boot disc, sometimes known as a system disc, is a disc that includes the boot partition.

You can also read about the Multilevel Queue Scheduling.

Bad block

Here, Most discs are packed with faulty blocks right out of the manufacturing and treated in a variety of ways. Because moving parts have short tolerances, discs are prone to errors. In this, the controller keeps a list of problematic blocks. Each faulty sector can be logically replaced with one of the spare sectors by the controller. Sector sparing or transfer is the term for this strategy. The data recovery process is triggered by a mild error. On the other hand, unrecoverable hard mistakes can result in data loss and need user intervention.

What does disk management do?

Disk management in an operating system provides tools to manage and control the storage devices, such as a computer system's hard disk drives (HDDs) or solid-state drives (SSDs). Some key tasks performed by disk management are:

1. Partitioning: Disk management allows users to create many partitions on a single disk. We can also resize the disk partitions as per the need without facing any data loss.

2. Formatting: Disk management provides options to format partitions with specific file systems, such as FAT32, NTFS, etc.
3. Disk Cleanup: Disk management tools also offer disk cleanup tools that help in removing unnecessary and junk files from the disk.
4. Disk Encryption: Some disk management tools provide disk encryption features to secure the data stored on disks.

Advantage of disk management in os

The advantages of disk management in OS are:

- **Storage Organization:** Disk management allows users to efficiently organize their storage devices and hard drives. It provides tools to create partitions, format drives, and perform the associated read and write operations. We can even resize the disks according to the need.

- **Improved Efficiency:** Disk management in an Operating System is useful in improving the performance of a computer system by freeing up space and preventing system lag. This enables faster access to files, thereby increasing the performance of the system.

- **Increased Security:** Disk management in OS enhances data security by storing files in a secure manner. Some disk management tools offer disk encryption features that allow users to encrypt their disks or specific partitions, adding an extra layer of security to protect sensitive data, thus preventing data loss or theft.

- **Better Backup:** With the help of disk management, we can store data in files that are organized in a logical format. This provides a better backup facility preventing any loss of data due to a power outage or any other reason.

- **Disk monitoring:** Disk management also offers monitoring and diagnostic capabilities. We can get information about disk health, including temperature, performance, and other attributes.

Disadvantages of disk management in os

The disadvantages of disk management in OS are:

- **Increased complexity:** Disk management can be complex for beginners with little technical knowledge. The complexity increases when there are multiple disk partitions already.

- **Risk of data loss:** If the Disk management tools are not used correctly, there can be a risk of data loss. For example, the user may delete the data of a disk by mistake while resizing or partitioning them.

- **Time-taking processes:** Some disk management operations, such as disk defragmentation or partition resizing, can be time-taking, specifically when dealing with large amounts of data.